

maktaba-web-local (المكتبة الناطقة, “The Speaking Library”) — The Complete Story, in Plain Language

A full, detailed, non-technical retelling of everything the system is and does

Author: Ayman Kazim Yousef · Department of Artificial Intelligence Engineering, AlSafwa University, Karbala, Iraq · ORCID 0009-0006-7409-9367 · kazi-mayemn@gmail.com

This document explains **maktaba-web-local** end to end, every part, in plain language — what it is, the problem it solves, how each piece works, why each choice was made, what is genuinely new versus standard, the experiments and their numbers, and where it honestly stops. It is written so a motivated non-specialist can follow it, while staying faithful to the real code. Figures are in figures/. Sources are listed where claims rest on outside work. It is the companion to the three research papers in ~/Desktop/14-6/ and to the algorithm catalog at ~/Desktop/maktaba_web_local_Algorithms_Catalog.md.

A note on honesty. This system is mostly an extremely well-engineered assembly of *known* building blocks, with a small number of *genuinely new* ideas concentrated in one place — how it decides whether a retrieved passage is relevant. Throughout, novel parts are marked ★, sound-but-known extensions ~, and standard textbook parts =. Nothing here is inflated; where a component is ordinary engineering, this document says so.

1. The 60-second summary (what the system is)

Imagine a private librarian that lives entirely on your own laptop. You give it your PDFs — textbooks, papers, notes, in Arabic or English. You ask it questions out loud or by typing, and it answers **from your books**, quotes which book it used, and replies in your own dialect (Iraqi Arabic). It never sends your books or your questions to the internet; everything happens on your machine. If your books genuinely don’t contain the answer, it says so and can fall back to its general knowledge or a labelled web search rather than inventing a source.

That is **maktaba-web-local**. Technically it is a *retrieval-augmented generation* (RAG) system: it **retrieves** the most relevant passages from your library, then asks a local language model to **generate** an answer grounded in those passages. It serves many users from one program (it is *multi-tenant*), and each user’s library is kept strictly separate.

The single most important new idea inside it answers a deceptively simple question: **“is any passage I found actually relevant enough to answer from?”** Getting that decision right — automatically, per user, with no labels and no internet — is the heart of the project’s research contribution. Around that core sit several other careful mechanisms (a cross-language dictionary that teaches itself, a fix for a subtle

multi-user search bug, and a safe Iraqi-dialect rewriter), and a great deal of solid offline engineering.

2. The problem this solves

People increasingly want to “chat with their documents.” The popular way to do this sends your documents to a cloud service. For a student in Karbala with private study material, an unreliable connection, a modest laptop, and content in Arabic, that approach fails on four counts at once:

- **Privacy.** Your books and questions leave your machine. For personal or sensitive material this is unacceptable; and there is no telemetry to “opt out of” if the data path never reaches a network in the first place.
- **Offline.** Cloud RAG needs the internet. maktaba runs with no connection — the language model, the search index, the embeddings, everything is local.
- **Cost and hardware.** Cloud APIs cost money per query; big GPUs are unavailable. maktaba targets a normal CPU laptop.
- **Language.** Most tools are tuned for English. maktaba handles Arabic and English together, including the fact that your Arabic question might only be answerable from an English book, and that you expect the answer in Iraqi dialect, not formal Arabic.

The deep difficulty is not “find some passages and feed them to a model” — that is easy. The difficulty is the **quality gate around it**: deciding when the found passages are good enough to trust, when to translate, when to fall back, and how to do all of that *per user* and *without labels*, on a machine with no GPU and no internet. That gate is where most of the engineering and all of the research novelty live.

3. The vocabulary, in plain words

- **RAG (retrieval-augmented generation).** Answer a question by first *retrieving* relevant text, then *generating* an answer that uses it. Keeps the model grounded in your documents instead of its memory.
- **Tenant.** One user. The system is *multi-tenant*: many users, one program, each user’s books fully isolated (`user_id` filters every step).
- **Chunk / passage.** A book is split into small pieces (a few hundred characters). Search and answering work on chunks, not whole books.
- **BM25 (lexical / keyword search).** A classic, fast formula that ranks passages by **word overlap** with the query, weighting rare words more. It matches *surface words*, so it cannot cross scripts (an Arabic query shares no letters with an English passage).
- **Embedding (dense / semantic search).** A neural model turns a passage into a list of numbers (a *vector*) so that passages with similar *meaning* land near each other, even across languages. Searching by vector nearness is *dense* or *semantic* search.
- **FAISS / HNSW.** A fast index for finding the nearest vectors among many.

- **RRF (Reciprocal Rank Fusion).** A simple, robust way to merge two ranked lists (keyword + semantic) into one.
 - **Cross-encoder / reranker.** A heavier neural model that reads the query and a passage *together* and outputs a relevance score. More accurate than the first-pass search, so it *reranks* the top candidates.
 - **Relevance gate.** The yes/no decision: is the best reranked passage relevant enough to answer from? It compares the top score to a **cutoff** τ .
 - **Calibration.** Choosing that cutoff correctly. The project's core contribution is choosing it **automatically, per tenant, with no labels**.
 - **Offline / on-device.** Everything runs on your computer; nothing is sent to a server.
 - **MSA vs dialect.** Modern Standard Arabic is the formal written language; Iraqi Arabic is how people actually speak. Language models answer in MSA by default; users want Iraqi.
-

4. How others have tried — and the gaps this fills

- **Cloud RAG assistants** (the common product) solve generation quality but break privacy, need the internet, cost money, and rarely handle Arabic dialects. maktaba's whole design is the offline, private, multilingual answer to this.
 - **A single hand-tuned relevance cutoff** (what almost every RAG system uses) is brittle: a reranker's scores shift with the topic mix, with the language (Arabic passages score lower than equally-relevant English ones), and — in a multi-user system — with *which user* is asking. One global number cannot be right for everyone. maktaba's self-calibrating gate replaces it (Section 8).
 - **Multi-tenant vector databases** isolate *which* vectors each user can search, but they leave the relevance cutoff a shared constant, and they handle isolation by pushing the user filter *into* the vector search. maktaba measures that the same crowding starvation hits *both* the keyword and the vector side whenever the filter is applied *after* a shared fetch, and shows the keyword (inverted-index) side has no in-search-filter escape hatch — so it gives each user their own keyword index (Section 10).
 - **Always-translate cross-lingual search** pays a translation cost on every query — too slow for an offline CPU. maktaba translates only when a same-language search genuinely fails, and it teaches itself a dictionary so it rarely needs to translate the same term twice (Section 9).
 - **Fine-tuning a model to speak a dialect** is infeasible offline and must be redone per model. maktaba leaves the model alone and rewrites its output into Iraqi afterward, safely (Section 11).
-

5. The architecture: the whole machine, end to end

The picture above is the whole system. Read it top to bottom. A request from the **browser** passes through **security middleware** and **guards** (rate limits, duplicate-request suppression, a circuit breaker that protects the model). It reaches **context.py**,

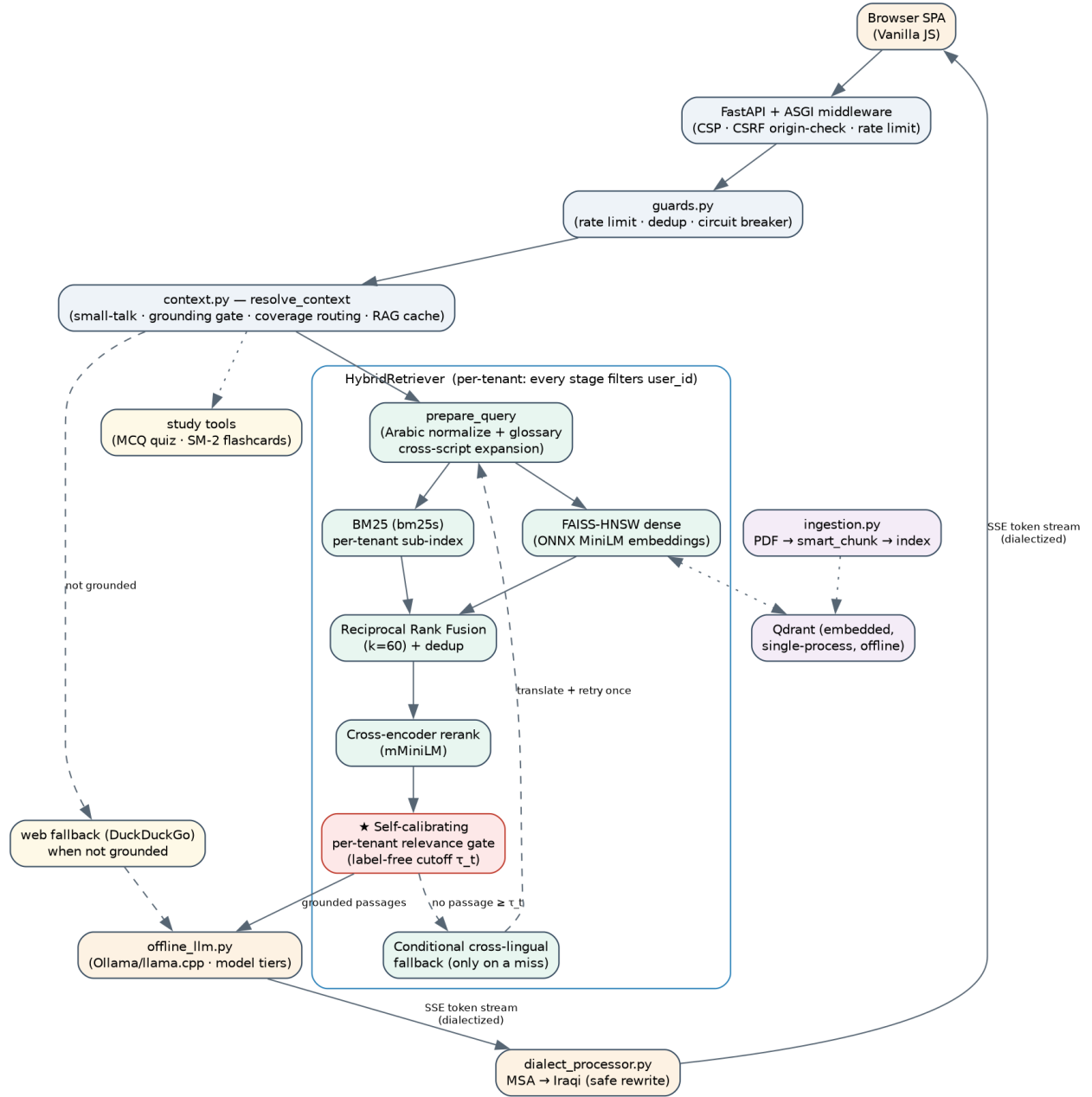


Figure 1: System architecture and request flow

the conductor: it handles small talk, decides whether the question is *grounded* in the books, and owns a short-lived cache. For a real question it calls the **HybridRetriever**.

Inside the retriever (the blue box) the question is first **normalized** and **expanded** with cross-language equivalents of any known terms. Two searches run in parallel — **BM25** (keywords, using a per-tenant sub-index) and **FAISS** (semantic vectors) — and their rankings are **fused** by RRF. A **cross-encoder reranks** the fused candidates, and then the ★ **self-calibrating relevance gate** applies the tenant’s own cutoff. If nothing clears the bar, a **conditional cross-lingual fallback** translates the query once and retries. The surviving passages go to the **offline language model**, whose answer is **rewritten into Iraqi dialect** and streamed back to the browser token by token. Books live in an **embedded Qdrant** vector store that runs *inside* the program (no separate server, no network); **ingestion** turns uploaded PDFs into indexed chunks in the background. On the side sit the **study tools** (quizzes, flashcards) and an optional labelled **web fallback**.

Three properties hold across the whole picture: it is **offline** (no step needs the internet), it is **per-tenant** (every step filters by `user_id`), and it **degrades gracefully** (if a heavy optional part — GPU, reranker, web, voice — is missing, the system still answers).

6. The journey of one question, step by step

1. **You ask.** “شئو الفرق بين المتغير والثابت؟” (“What’s the difference between a variable and a constant?”). The browser opens a streaming connection.
2. **Gatekeeping.** Middleware checks security headers and rate limits; guards drop duplicate sends and trip a circuit breaker if the model is failing.
3. **Small talk?** `context.py` checks whether this is a greeting (“hi”, “شلونك”) — if so it answers directly in Iraqi, no search.
4. **Query preparation.** The Arabic is normalized (accents stripped, letter forms unified). The **glossary** notices “متغير/ثابت/برمجة” and appends “variable / constant / programming” so an English programming book becomes findable.
5. **Two searches.** BM25 keyword search runs on *your* sub-index; FAISS semantic search runs on the shared vector index but filters to your books *during* the search. Each returns its top candidates (the system over-fetches to be safe).
6. **Fuse and rerank.** RRF merges the two lists; the cross-encoder rereads each candidate with the query and scores it.
7. **The gate decides.** The system compares the best score to *your* calibrated cutoff τ . Above it → answer from the passages. Below it → either translate-and-retry once, or answer from general knowledge / labelled web, saying so.
8. **Generate.** The local model writes an answer grounded in the kept passages, citing the book.
9. **Dialectize.** The answer (in formal Arabic) is rewritten into Iraqi — هسه, ماکو, شلون — without ever corrupting a word, and code/formulas are left untouched.
10. **Stream.** Tokens arrive in the browser as they are produced; the conversation is saved locally.

Every one of these steps runs on your machine.

7. The retrieval engine, in plain words

The retriever’s job is to bring the *right* passages to the front. It uses two complementary searches because each is blind where the other sees:

- **Keyword search (BM25)** is precise about exact words and rare terms but literal — it cannot tell that “car” and “automobile” mean the same thing, and it cannot cross scripts. It is implemented with **bm25s**, a fast modern library.
- **Semantic search (embeddings + FAISS)** captures meaning and crosses languages, but can be vague — it sometimes returns passages that are “about the same area” without actually answering.

Fusing them with **RRF** keeps the strengths of both: a passage that ranks well in *either* list rises. Then the **cross-encoder reranker** — a small multilingual model that reads query and passage together — gives a sharp final relevance score to the top handful. The system deliberately **over-fetches** (pulls more candidates than it will keep) so that a short book of 22 chunks is not crowded out of the candidate pool by a 1,400-chunk book before reranking.

All of this — BM25, dense retrieval, RRF, cross-encoder reranking, over-fetch — is **standard, well-understood IR machinery (=)**. The project’s contributions are not here; they are in the *decision made on top of this stack* (the gate, Section 8), in a *correctness bug in the keyword half under many users* (Section 10), and in the *cross-language expansion* feeding the front of it (Section 9).

8. The self-calibrating relevance gate — the heart of the invention (★ the one genuinely novel idea)

Here is the deceptively simple decision again: after searching and reranking, **is the best passage relevant enough to answer from?** If yes, ground the answer in it. If no, the honest move is to *not* force an off-topic passage into the model (which causes confident, wrong, mis-cited answers) — instead answer from general knowledge or fall back.

This reduces to comparing the best reranked score to a **cutoff** τ . Almost every RAG system uses one hand-picked number for τ . That is brittle for three compounding reasons:

1. **Topic shift.** A library on one tight subject makes everything look somewhat relevant (scores run high); a broad library runs lower. The “relevant” band moves with the corpus.
2. **Language shift.** The multilingual reranker scores an equally-relevant Arabic passage *lower* than an English one. A cutoff tuned on English silently over-filters Arabic.
3. **Tenant shift.** Every user owns a different library, so every user’s score geometry differs. One global number over-filters some users and under-filters others.

The idea (and why it is clever). Instead of guessing τ or learning it from labelled data (which a private, brand-new, offline library does not have), derive it from the

scorer’s own behaviour on the user’s own books, with no labels:

- Take a passage and use its own first few words as a pretend query. Score that query against **the very same passage**. This is a **guaranteed-relevant** example — an *upper anchor* for what “relevant” looks like in this library, for this scorer, in this language.
- Score the same pretend query against a chunk from a **different book**. This is almost-certainly **irrelevant** — a *lower anchor*.
- Do this for a handful of passages. Now you have two clouds of scores: a “relevant” band (high) and an “irrelevant” band (low). If they separate, put the cutoff **in the gap between them**, a quarter of the way up from the irrelevant band (favouring recall). If they don’t separate, use a safe default.

That’s it: the cutoff is read off the **geometry of the scorer’s own scores** on the user’s own corpus. No labels, no feedback, no internet, works on a brand-new user with zero history, and it is computed entirely on the device. It is also **scorer-agnostic**: the identical procedure works on the heavy cross-encoder *or* on the light embedding model (cosine similarity), so it runs even on a device too weak for the cross-encoder.

What it buys, measured. On the real library, the label-free gate reaches **F1 0.968** versus **0.828** for the previous hand-tuned default — lifting precision from **0.71 to 0.94** while keeping recall at 1.0 (it removes the off-topic acceptances that cause hallucinated, mis-attributed answers). Run unchanged over four different user libraries, it produces four *different* cutoffs — direct proof that one global number cannot fit everyone. And as an unexpected bonus it derives a *higher* cutoff for the Arabic-heavy library, partially compensating the reranker’s per-language bias on its own.

An under-used twist (the strongest remaining AI angle). Because the same calibration runs on *two* different scorers (the cross-encoder and the embedding model), the **disagreement** between the two derived cutoffs is itself a free, label-free signal of how trustworthy each scorer is for that particular library. This is implemented and already shows a real effect (one mixed-language library flips which scorer ranks it strictest), but it is not yet developed into its own method — a promising direction.

This gate is the **one genuinely novel (★) AI idea** at the centre of the system, and it is already written up as the flagship paper and a patent disclosure.

9. The self-teaching cross-language dictionary (★~ partially novel)

The problem. Your Arabic question may only be answerable from an English book. Keyword search matches letters, so an Arabic query finds *nothing* in an English book even when it is the perfect source. Translating the whole query with the language model fixes it but costs seconds of CPU time *every* query — too slow.

The mechanism. Keep a small bilingual **glossary** of subject terms (308 Arabic→English and 280 English→Arabic pairs across programming, math, ML, economics, security, databases). Before searching, silently append the other-language equivalent of any term the system recognizes (“متغير” → also search “variable”). This is instant, needs no model, and can only *help* (a term not in the glossary is simply left alone).

The new part: it teaches itself. A fixed dictionary always misses the word you

need. So the system **grows its own**: the one time a search finds nothing and the model is asked to translate the query, the resulting word-pair is **saved to a local file**. From then on, that term is expanded instantly, with no model call, forever. The system literally **learns the user’s vocabulary from its own failures** — continually, offline, with no model retraining (it just appends a line to a JSON file). This is the project’s feasible form of “continual learning”: adapting the *retrieval layer*, not the frozen chat model.

Measured. With the glossary, Arabic→English book-level retrieval rises from **0.0 to 1.0** (zero is the floor — letters can’t cross scripts); and three brand-new held-out terms go from **0.0 to 1.0** retrieval after a single self-taught acquisition each. (Honestly: English→Arabic was already near-perfect because Arabic technical books embed English words like “variable” — so the glossary’s gain is concentrated on the Arabic→English direction, which we report rather than average away.) Dictionary-based cross-language search and query expansion are decades old (~); the **continual, label-free, no-retraining acquisition loop** is the new part (★ in spirit, “partially novel” against the closest 2025–2026 work).

10. The multi-user keyword-search bug, and its fix (★~ partially novel)

The discovery. In a multi-user system, the easy design puts *everyone’s* passages in one keyword index, retrieves the global top-N, then keeps only the asker’s rows. This hides a real bug: when one user owns almost the whole library (in the live system, **3,725 of 3,783 attributed passages — 98.5%**), that user’s passages fill the global top-N, so a minority user’s relevant passages are thrown away *before* the “keep only my rows” filter. The minority user is **starved** of their own results.

The honest nuance (a strength, not a weakness). This does not happen for every query. It happens specifically when the minority user’s question uses **ordinary words that the dominant user’s books also use heavily**. We proved this with a control: for questions on vocabulary *distinct* from the dominant library, recovery stays ~0.91–0.97 regardless of size; for *shared* vocabulary it collapses to **0.46** at the real 98.5% skew (recovering under half of the correct passages), and the candidate pool halves (4.7 → 2.1 of 5). We even decomposed *why*: **0.39** of the loss is crowding (the dominant user’s passages occupy the top), and **0.14** is “statistics capture” (the keyword formula’s word-importance weights are set by the dominant user’s books).

The fix. Give each user their own small keyword index, built the first time they search and reused after (about **2.7 milliseconds** to build, cached in memory). Then ranking and word-statistics are computed within the user’s own books — recovery returns to 1.0, regardless of how large anyone else grows.

The deeper point. It is tempting to assume the *semantic* (vector) search escapes this — but we **measured** that it does not, when it filters the same way (fetch a global batch, then keep the asker’s rows): at the real 98.5 % skew its recovery collapses from 1.0 to **0.32**, even *below* the keyword side’s 0.46 (Fig. P1-d). Crowding hits both. The genuine asymmetry is *architectural*: a vector index **can** push the “is this my book?” test *into* the search itself (the Filtered-DiskANN / ACORN technique), continuing until it has enough of the right user’s results — whereas a shared keyword (inverted) index has no equivalent, short of giving each user their own index. So the per-user index is the

keyword side’s version of that in-search filter. Naming and measuring this starvation — and showing it is really *post-filter-after-shared-fetch* vs *filter-in-search*, not simply keyword vs vector — is the contribution (found only in practitioner forums before, never in a peer-reviewed paper). The per-user index itself is standard engineering (~); the **diagnosis** is the new part.

11. ~ The safe Iraqi-dialect rewriter

The problem. The model answers in formal Arabic (MSA); Iraqi students want Iraqi (هسه = now, ماكو = there isn’t, شلون = how). Retraining the model per dialect is infeasible offline; just asking it to “speak Iraqi” is unreliable.

The mechanism. Leave the model alone and **rewrite its answer afterward** with a fixed, deterministic layer: 22 morphological rules (verb tense, دخلنا→دعنا, etc.) + a 321-entry word-substitution map (hand-built using real Iraqi datasets and the Georgetown Dictionary of Iraqi Arabic) + a gentle spacing fixer. Because it runs *after* the model and depends on nothing about it, it works with *any* model and is fully reproducible. Code blocks and formulas are passed through untouched.

The real lesson — “safe rewriting.” Arabic words have no internal spaces but rich prefixes, and many dialect markers (لا, على, مع, the future س) are also *pieces of ordinary words*. A careless rule that “splits on على” or “rewrites a leading س” shatters normal words: الاقتصاد (“the economy”) → “سيارة”, الاقتصاد (“car”) → “راح يارة”. An earlier version did exactly this — the code records it corrupted **17 of 21** and **14 of 18** common words — so those rules were removed. The deployed rewriter uses only **boundary-guarded** rules that *cannot* change a letter inside a word, trading some dialect coverage for **zero** corruption. Measured: the deployed layer corrupts **0/21** and **0/18** where a naive baseline corrupts **21/10**; it converts all 15 test sentences into recognizable Iraqi (~54% of tokens), is idempotent, and preserves code. The components (rule-based dialect conversion, lexicons) exist already (=/~); the new parts are the *deterministic, post-hoc, model-agnostic* placement and the documented *safe-rewriting methodology*. (Honest gap: no human fluency study yet — that is the main future step.)

12. = Embeddings and offline inference

To do semantic search, every chunk and every query must become a vector. maktaba uses a multilingual MiniLM embedding model run through **ONNX Runtime** (a fast, portable inference engine), on CPU by default with optional GPU. Three pieces of careful engineering (all standard MLOps, none novel):

- **An FP16 query cache** stores recent query vectors at half precision (768 bytes instead of 1536), with negligible accuracy loss.
- **Crash-isolated GPU probing:** before trusting the GPU, the system tries it in a separate throwaway process, so that a graphics-driver crash cannot take down the server; it also only uses the GPU if enough memory is free, and halves its batch size if it runs out.
- **A native C++ chunker** (smart_chunk) splits text on Arabic and Latin sentence boundaries at proper character boundaries; a pure-Python splitter is the fallback.

An important honesty note. The repository contains C++ files for a custom quantized neural engine (hand-written matrix kernels, an INT8/INT4 quantizer). After a line-level audit: **this code is dead** — it is not compiled on the Linux build, not exported to Python, and never called; all inference goes through ONNX Runtime. It is scaffolding for a future phone/NPU port. It must **not** be presented as a working contribution. The one native piece that genuinely runs is the chunker.

13. = Ingestion, indexing, and the offline vector store

When you upload a PDF, a background process extracts its text (PyMuPDF), splits it into chunks, embeds them, and stores them in **Qdrant** — a vector database that, crucially, runs **embedded inside the program** (no separate server, no network), matching the offline goal. Progress is tracked so a crash can resume. The keyword index and the semantic index are cached to disk and rebuilt incrementally when you add a book (re-tokenizing in seconds instead of rebuilding from scratch in ~30 seconds); a cheap count-comparison detects when a cache is stale. The trade-off of embedded Qdrant — only one process may open the data at a time — is documented and accepted for a single-user-per-machine setting; a local server mode exists if true concurrency is ever needed. All standard, sturdy engineering.

14. = The language-model layer

The answer itself is written by a local large language model. *maktaba* speaks to several possible providers through one interface, trying them in order: llama.cpp → Ollama → (optionally) Groq/OpenAI/Anthropic. By default it is fully local (Ollama running qwen2.5). It offers **model tiers** — light (3B), balanced (7B), max (14B) — and recommends one based on how much RAM you have (~0.7 GB per billion parameters), and can hot-swap between them without restarting (unloading the previous model's memory first). It strips the model's internal "thinking" tags and any stray non-Arabic preamble. A small block of curated Iraqi examples is injected into the system prompt, **scoped to greetings only** — an earlier unscoped version made the model sprinkle "شلونك؟" into the middle of technical answers, so the scoping is deliberate. All standard provider-abstraction and config engineering.

15. = Study tools

Because the target user is a student, two learning features sit on top of the same library:

- **Quiz (MCQ) generation.** The model is asked for multiple-choice questions in a strict JSON shape; a robust parser salvages truncated output, rejects near-duplicate options and meta-questions ("according to the passage..."). Notably, quiz generation **deliberately bypasses the relevance gate** — for whole-book coverage you want broad sampling, not the narrow "is this relevant to a query" filter. A sensible design choice, not a research claim.
- **Flashcards with spaced repetition.** Review scheduling uses **SM-2**, the classic SuperMemo-2 formula (1990). Standard and correctly attributed; no novelty claimed.

16. ~ Security and privacy — by architecture, not by policy

Privacy here is structural: there is nothing to “opt out of” because the data path never reaches a network. On top of that offline guarantee sit standard, careful protections:

- **Passwords** hashed with bcrypt; a *dummy* bcrypt is run even for unknown user-names so an attacker can’t tell which accounts exist from response timing.
- **Session tokens** are random, stored only as SHA-256 hashes at rest, and delivered as HttpOnly cookies (JavaScript can’t read them); no JWT.
- **Web middleware**: content-security-policy headers, a CSRF origin check, per-IP rate limiting (which skips static files so the page can load).
- **Reliability guards**: a circuit breaker that stops hammering a failing model, duplicate-request suppression, and per-user rate limits.
- **Strict per-tenant isolation**: every query, file path, and database row is filtered by `user_id`; the system never reads or deletes across users.

The dummy-hash timing defense is a nice touch (~); the rest is solid textbook security (=).

17. The experiments and their numbers, in plain language

Every number in the three research papers comes from a deterministic script run on the **real** library (3,838 indexed passages — two named users owning 3,725 and 58 passages, plus 55 older unattributed chunks — on CPU) — re-runnable, no fabrication. The figures referenced below are in figures/.

- **The gate (flagship)**. F1 **0.968** vs **0.828** for the old fixed cutoff; precision **0.71** → **0.94** at recall 1.0; four users → four different cutoffs.
- **Keyword starvation (Paper 1)**. For shared-vocabulary minority queries, recovery of the correct results collapses from **0.91** to **0.46** as one user grows to **98.5** % of the library, while the distinct-vocabulary control stays flat at ~0.91–0.97 (Fig. P1-a) — proving the cause is *shared vocabulary*, not size; the candidate pool halves (Fig. P1-b). Over-fetching hides the symptom but not the loss (Fig. P1-c); the loss decomposes into 0.39 crowding + 0.14 statistics-capture; and a measured *dense* baseline starves too (Fig. P1-d). The per-user index restores it to 1.0 at ~2.7 ms.
- **Cross-language glossary (Paper 2)**. Arabic→English retrieval **0.0** → **1.0** with the glossary; held-out terms **0.0** → **1.0** after one self-taught acquisition each (Figs. P2-a, P2-b).
- **Dialect rewriter (Paper 3)**. The deployed rewriter corrupts **0** of 21 and **0** of 18 common words where a naive baseline corrupts **21** and **10** (Fig. P3); 100 % of test sentences dialectized; idempotent; code-preserving.

18. What is genuinely new vs standard (the honest ledger)

- ★ **Genuinely new (the real IP)**: the self-calibrating per-tenant relevance gate, its scorer-agnostic form, and the (under-developed) dual-scorer calibration-disagreement signal. This is the project’s contribution to knowledge.

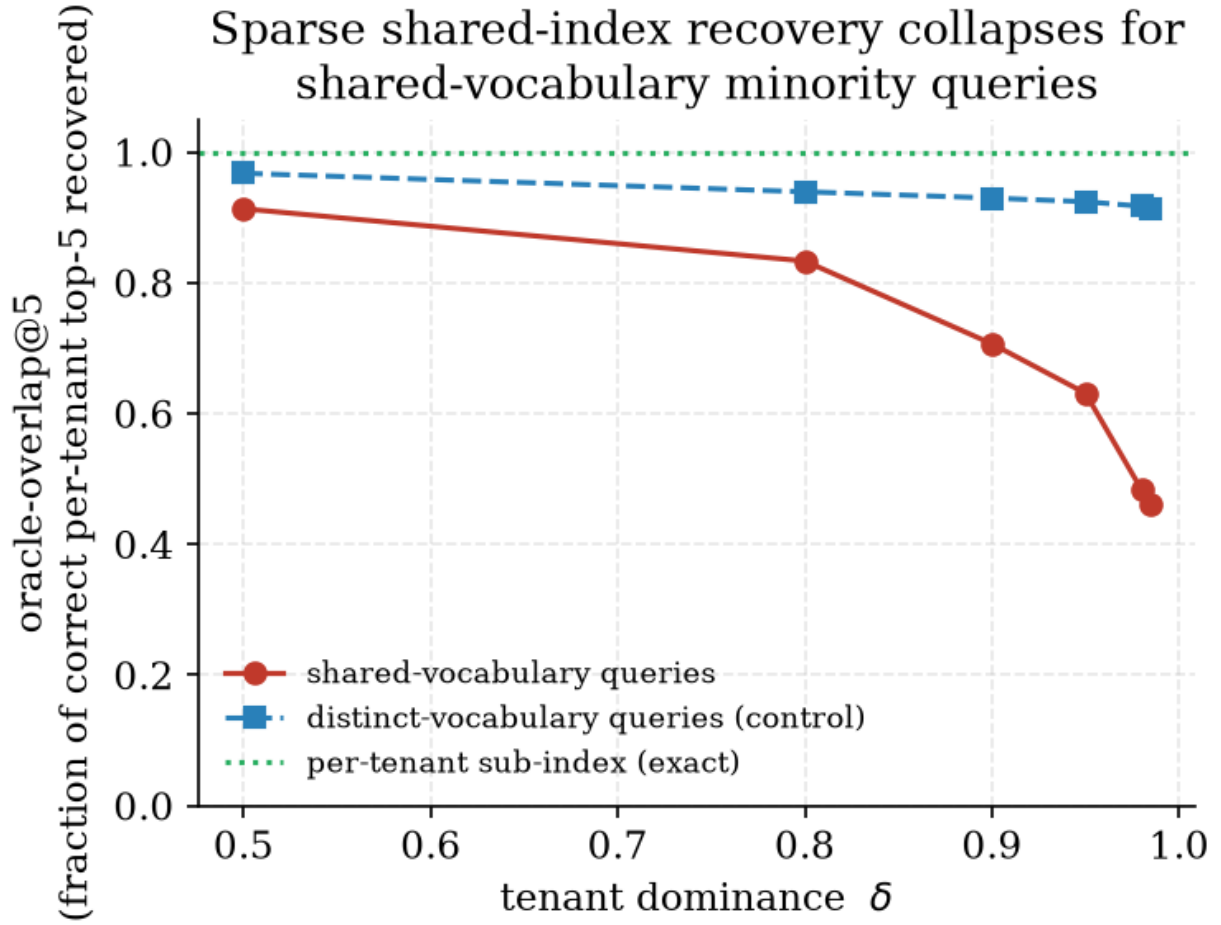


Figure 2: P1-a — oracle-overlap@5 collapses for shared-vocabulary minority queries (red); the distinct-vocabulary control is flat (blue); the per-tenant sub-index is exact (green).

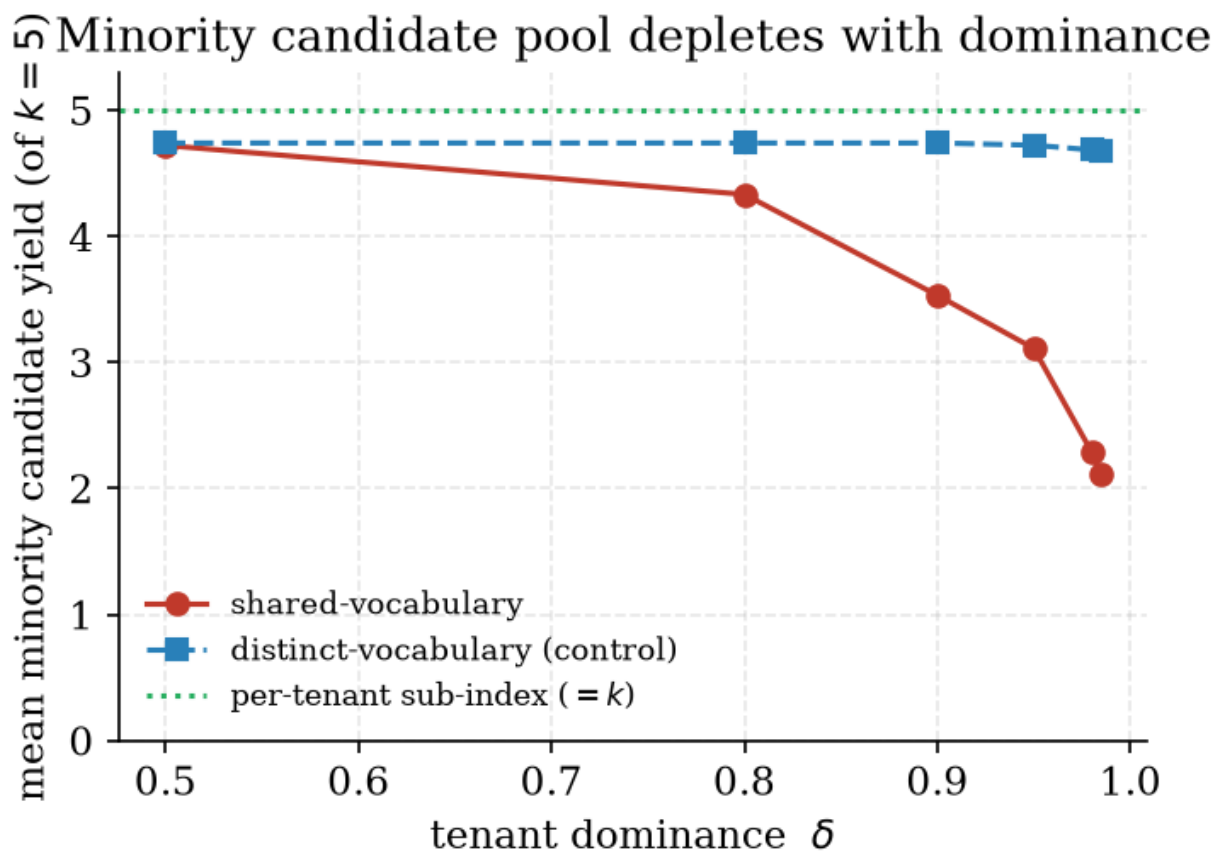


Figure 3: P1-b — the minority candidate pool (yield of 5) depletes with dominance.

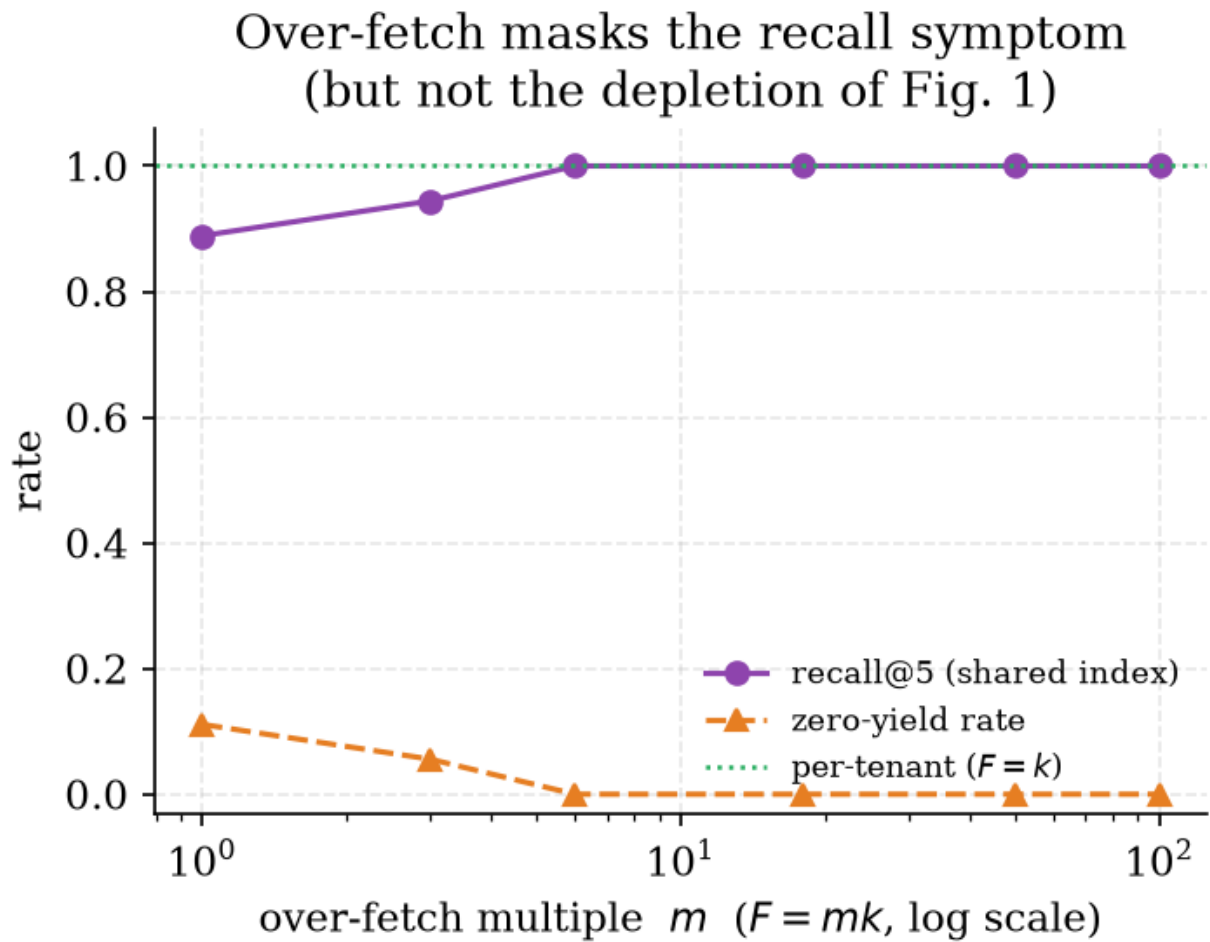


Figure 4: P1-c — over-fetch raises target recall but does not restore the candidate-pool depletion.

Crowding starves BOTH sparse and dense post-filtering;
per-tenant indexing fixes both

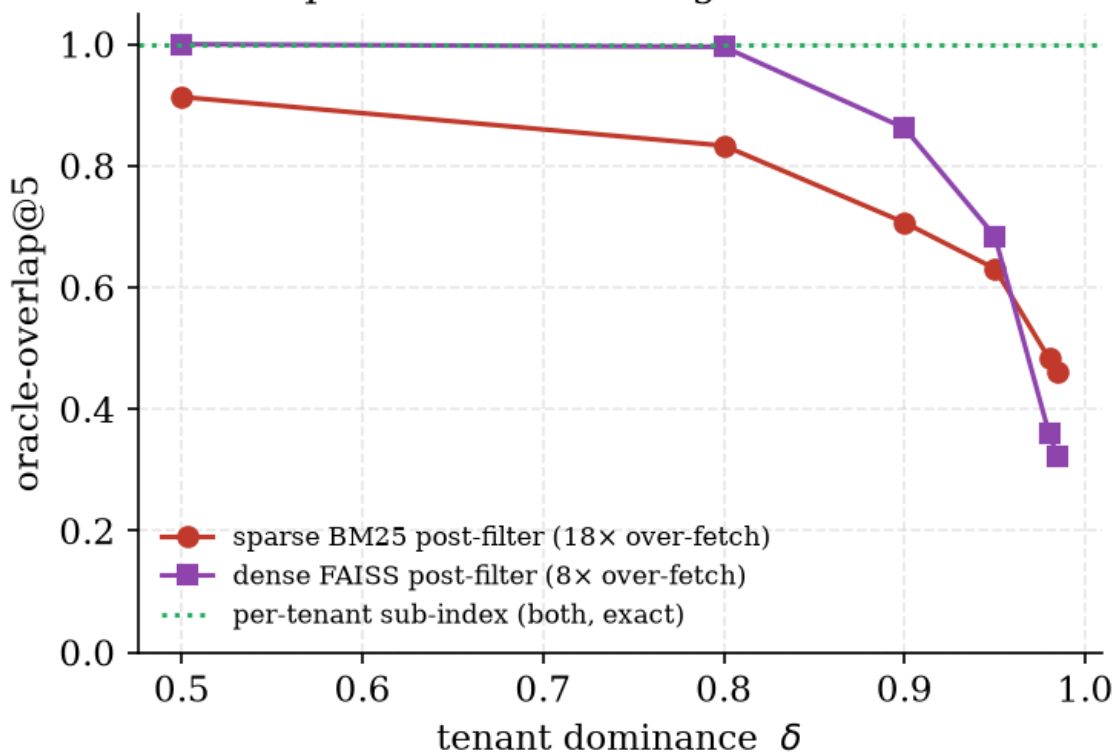


Figure 5: P1-d — measured: crowding starves both sparse and dense post-filtering; the per-tenant index fixes both.

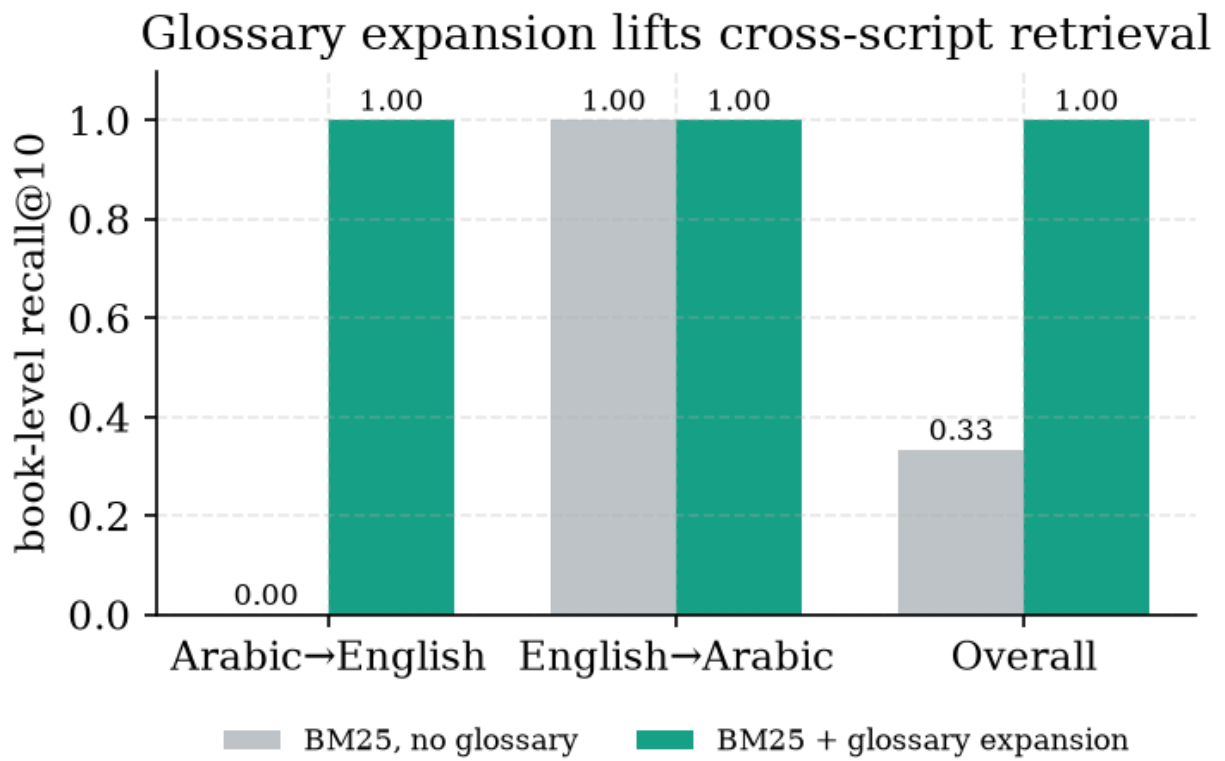


Figure 6: P2-a — glossary expansion lifts cross-script book-level recall (Arabic→English 0.0→1.0).

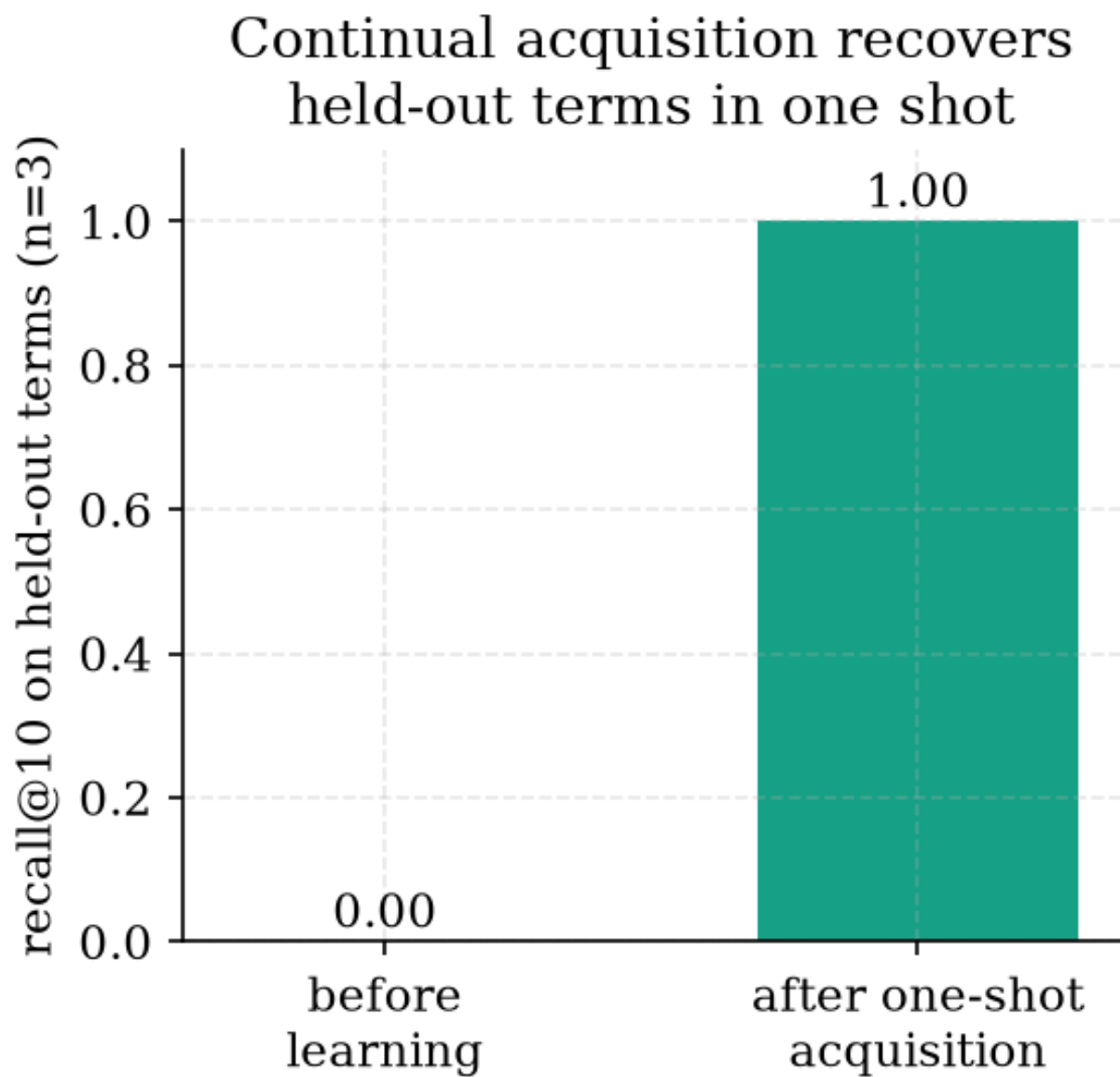


Figure 7: P2-b — continual learning recovers held-out terms (0.0→1.0) after one-shot acquisition.

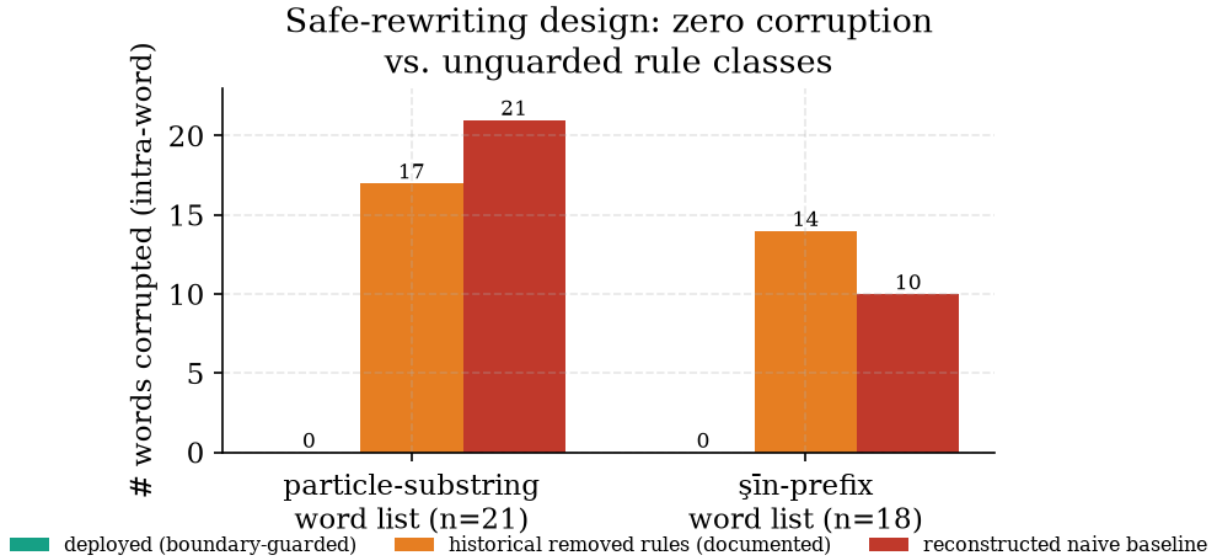


Figure 8: P3 — safe rewriting: zero corruption vs. unguarded rule classes.

- **~ Sound, focused contributions (the three new papers):** the keyword-starvation diagnosis + per-tenant index; the continual self-teaching cross-language glossary; the safe Iraqi-dialect rewriting methodology. Each is a real, defensible systems/IR/NLP contribution, honestly “partially novel” — the building blocks exist; the specific synthesis or diagnosis is new.
- **= Standard, used as-is:** BM25, dense retrieval, RRF, cross-encoder reranking, HyDE, RAPTOR, multi-query, ONNX/INT8, embedded Qdrant, SM-2, MCQ validation, the security stack, model tiering.
- **[!] Present but NOT a contribution (must not be claimed):** the native C++ quantizer/transformer kernels (dead code on this build); the MARBERT dialect detector (disabled). The offline voice/XTTS layer is real but never evaluated — an unexplored angle, not yet a claim.

19. Where it honestly stops (limitations)

- The research evaluations are **small, single-machine micro-benchmarks** (two named tenants, passage-derived probe queries) — they characterize *mechanisms*, not large-scale deployment quality.
- No **end-to-end answer-quality / hallucination** user study yet; the gate’s benefit is argued from precision, not measured on generated answers.
- The dialect rewriter has **no human fluency study** yet — it is proven *safe* and *broad*, not yet judged *natural* by Iraqi speakers.
- The glossary’s continual loop is demonstrated *by construction*; the deployed learned dictionary has not yet accumulated at scale in real use.
- The system is a careful assembly of mostly-known parts; its novelty is concentrated, not pervasive — this document marks exactly where.

20. How to reproduce every number

Everything is deterministic and local. With the project's Python environment:

```
cd ~/Desktop/14-6/_experiments
PY=~/.venv/bin/python
$PY exp_p1_starvation.py      # P1 starvation curves
$PY exp_p1b_ablation.py      # P1 crowding-vs-IDF + CIs
$PY exp_p1c_dense.py         # P1 measured dense baseline
$PY exp_p2_glossary.py       # P2 cross-script + continual
$PY exp_p2b_ablation.py      # P2 glossary-size ablation
$PY exp_p3_dialect.py        # P3 safety/coverage/idempotence
$PY exp_p3b_dialectness.py   # P3 automatic dialectness (ALDi)
$PY exp_perf.py              # latency + memory
$PY make_figures.py          # regenerate all figures
```

Each writes its results to `_experiments/results/*.json`; the figures script regenerates every figure. Because the procedures are label-free and deterministic, pointing them at the same corpus yields the same numbers.

Appendix A — Component & file map

Subsystem	Key files	Role
Entry / wiring	backend/server_backend.py	routers, middleware, lifespan, system prompt
Shared state	backend/core/state.py	singletons (retriever, books dir)
Conductor	backend/core/context.py	grounding gate, coverage routing, RAG cache, web fallback
Retrieval	backend/rag/hybrid_retriever.py	BM25 + FAISS + RRF + rerank + per-tenant sub-index + translate
Relevance gate	backend/rag/relevance_gate.py	self-calibrating per-tenant cutoff
Reranker	backend/rag/reranker.py	cross-encoder scoring
Glossary	backend/rag/glossary.py	cross-script expansion + continual learning
Embeddings	backend/rag/native_embeddings.py	ONNX vectors, FP16 cache, GPU probing
Expansion	backend/rag/hyde.py, multi_query.py, raptor.py	optional retrieval aids
Vector store	backend/database/vector_db.py	embedded Qdrant + FAISS index
Ingestion	backend/core/ingestion.py	PDF → chunk → index
LLM	backend/llm/offline_llm.py	providers, tiers, streaming
Dialect	backend/dialect/dialect_processor.py	MSA→Iraqi safe rewrite
Auth/guards	backend/auth/auth.py, backend/core/guards.py, backend/api/middleware.py	security + reliability
Study	backend/quiz.py, backend/srs.py	MCQ + spaced repetition

Subsystem	Key files	Role
Native	native_engine/src/document_optimizer.cpp	Arabic-aware chunking (the rest is dead code)

Appendix B — How the parts talk (data flow)

server_backend wires routers via a factory pattern (no circular imports) and shares singletons through state.py. A chat request flows middleware → guards → context → hybrid_retriever → (gate) → offline_llm → dialect_processor → SE. Ingestion writes to Qdrant on a background thread and signals the retriever to rebuild its caches. The gate reads the tenant’s corpus (via the per-tenant index) to compute its cutoff and caches it under a (tenant, scorer, corpus-state) key, so it recomputes only when the corpus changes.

Appendix C — The key algorithms as step-by-step recipes

Self-calibrating gate (per tenant): (1) sample chunks from the tenant’s books; (2) for each, score its leading words against itself (relevant band) and against another book’s chunk (irrelevant band); (3) if the bands separate, set τ a quarter of the way up the gap, else use a default; (4) cache τ ; recompute only when the corpus changes; (5) at query time, accept the top passage iff its score $\geq \tau$, else translate-and-retry or fall back.

Per-tenant keyword sub-index: (1) on a tenant’s first query, filter the corpus to that tenant *before* indexing; (2) build a small BM25 index over only those chunks; (3) cache it keyed by corpus identity+length; (4) reuse until the corpus changes; (5) retrieve top-k directly — no global top-N, no post-filter, no statistics distortion.

Continual glossary: (1) before search, append known other-language equivalents of query terms; (2) search; (3) if nothing is found and the query is a single term, translate it once with the model; (4) save the pair to the local dictionary; (5) thereafter expand that term instantly with no model call.

Safe dialectize: (1) stash code spans; (2) apply boundary-guarded morphological rules; (3) apply the length-sorted substitution map (longest phrases first), each rule unable to touch letters inside a word; (4) fix spacing without splitting words; (5) restore code spans.

Appendix D — Plain-language glossary

Tenant = one user. **Chunk** = a small passage. **BM25** = keyword search by word overlap. **Embedding** = a meaning-vector. **FAISS/HNSW** = fast nearest-vector index. **RRF** = merge two ranked lists. **Cross-encoder/reranker** = a model that scores a (query, passage) pair. **Relevance gate** = the accept/reject decision. τ (**tau**) = the relevance cutoff. **Calibration** = choosing τ correctly. **MSA** = formal Arabic; **dialect** = spoken (Iraqi) Arabic. **RAG** = retrieve-then-generate. **Offline/on-device** = runs entirely on your computer. **Multi-tenant** = many isolated users in one program. **Over-fetch** = pull more candidates than you keep. **Continual learning** = improving over time (here, by growing a dictionary, not retraining a model).

End of the complete plain-language explainer. Companion research papers and their verified sources are in ~/Desktop/14-6/; the algorithm catalog is at ~/Desktop/maktaba_web_local_Algorithms_Catalog.md.